10 Virtual Functions

Chapter Objectives

10.1 Pointers to derived classes

10.2 Introduction to virtual functions

10.3 More about virtual functions

10.4 Applying polymorphism

THIS chapter examines another important aspect of C++: the virtual function. What makes virtual functions important is that they are used to support run-time polymorphism. Polymorphism is supported by C++ in two ways. First, it is supported at compile time, through the use of overloaded operators and functions. Second, it is supported at run time, through the use of virtual functions. As you will learn, run-time polymorphism provides the greatest flexibility.
At the foundation of virtual functions and run-time polymorphism are pointers to derived classes. For this reason this chapter begins with a discussion of such pointers.

Review Skills Check

Before proceeding, you should be able to correctly answer the following questions and do the exercises.

1. Create a manipulator that causes numbers to be displayed in scientific notation, using a capital E.
2. Write a program that copies a text file. During the copy process, convert all tabs into the correct number of spaces.
3. Write a program that searches a text file for a word specified on the command line. Have the program display how many times the specified word is found. For simplicity, assume that anything surrounded by whitespace is a word.
4. Show the statement that sets the put pointer to the 234th byte in a file linked to a stream called **out**.
5. What functions report status information about the C++ I/O system?
6. Give one advantage of using the C++ I/O functions instead of the C-like I/O system.

10.1 POINTERS TO DERIVED CLASSES

Although Chapter 4 discussed C++ pointers at some length, one special aspect was deferred until now because it relates specifically to virtual functions. The feature is this: A pointer declared as a pointer to a base class can also be used to point to any class derived from that base. For example, assume two classes called **base** and **derived**, where **derived** inherits **base**. Given this situation, the following statements are correct:

```
base *p; // base class pointer

base base_ob; // object of type base
derived derived_ob; // object of type derived

// p can, of course, point to base objects
p = &base_ob; // p points to base object

// p can also point to derived objects without error
p = &derived_ob; // p points to derived object
```

As the comments suggest, a base pointer can point to an object of any class derived from that base without generating a type mismatch error.
Although you can use a base pointer to point to a derived object, you can access only those members of the derived object that were inherited from the base. This is because the base

pointer has knowledge only of the base class. It knows nothing about the members added by the derived class.
While it is permissible for a base pointer to point to a derived object, the reverse is not true. A pointer of the derived type cannot be used to access an object of the base class. (A type cast can be used to overcome this restriction, but its use is not recommended practice.)
One final point: Remember that pointer arithmetic is relative to the data type the pointer is declared as pointing to. Thus, if you point a base pointer to a derived object and then increment that pointer, it will not be pointing to the next derived object. It will be pointing to (what it thinks is) the next base object. Be careful about this.

EXAMPLE

1. Here is a short program that illustrates how a base class pointer can be used to access a derived class:

```
// Demonstrate pointer to derived class.
#include <iostream>
using namespace std;

class base
{
    int x;
public:
    void setx(int i) { x = i; }
    int getx() { return x; }
};

class derived : public base
{
    int y;
public:
    void sety(int i) { y = i; }
    int gety() { return y; }
};

int main()
{
    base *p; // pointer to base type
    base b_ob; // object of base
    derived d_ob; // object of derived

    // use p to access base object
    p = &b_ob;
    p->setx(10); // access base object
    cout << "Base object x: " << p->getx() << '\n';

    // use p to access derived object
    p = &d_ob; // point to derived object
    p->setx(99); // access derived object

    // can't use p to set y, so do it directly
    d_ob.sety(88);
```

```
    cout << "Derived object x: " << p->getx() << '\n';
    cout << "Derived object y: " << d_ob.gety() << '\n';

    return 0;
}
```

Aside from illustrating pointers to derived classes, there is no value in using a base class pointer in the way shown in this example. However, in the next section you will see why base class pointers to derived objects are so important.

EXERCISE

1. On your own, try the preceding example and experiment with it. For example, try declaring a derived pointer and having it access an object of the base class.

10.2 INTRODUCTION TO VIRTUAL FUNCTIONS

A *virtual function* is a member function that is declared within a base class and redefined by a derived class. To create a virtual function, precede the function's declaration with the keyword **virtual**. When a class containing a virtual function is inherited, the derived class redefines the virtual function relative to the derived class. In essence, virtual functions implement the "one interface, multiple methods" philosophy that underlies polymorphism. The virtual function within the base class defines the *form* of the *interface* to that function. Each redefinition of the virtual function by a derived class implements its operation as it relates specifically to the derived class. That is, the redefinition creates a *specific method*. When a virtual function is redefined by a derived class, the keyword **virtual** is not needed.
A virtual function can be called just like any other member function. However, what makes a virtual function interesting-and capable of supporting run-time polymorphism-is what happens when a virtual function is called through a pointer. From the preceding section you know that a base class pointer can be used to point to a derived class object. When a base pointer points to a derived object that contains a virtual function and that virtual function is called through that pointer, C++ determines which version of that function will be executed based upon the *type of object being pointed to* by the pointer. And, this determination is made at *run time*. Put differently, it is the type of the object pointed to at the time when the call occurs that determines which version of the virtual function will be executed. Therefore, if two or more different class are derived from a base class that contains a virtual function, then when different objects are pointed to by a base pointer, different versions of the virtual function are executed. This process is the way that run-time polymorphism is achieved. In fact, a class that contains a virtual function is referred to as a *polymorphic class*.

EXAMPLES

1. Here is a short example that uses a virtual function:

```
// A simple example using a virtual function.
#include <iostream>
using namespace std;

class base
{
public:
    int i;
```

```
    base(int x) { i = x; }
    virtual void func()
    {
        cout << "Using base version of func(): ";
        cout << i << '\n';
    }
};

class derived1 : public base
{
public:
    derived1(int x) : base(x) {}
    void func()
    {
        cout << "Using derived1's version of func(): ";
        cout << i*i << '\n';
    }
};

class derived2 : public base
{
public:
    derived2(int x) : base(x) {}
    void func()
    {
        cout << "Using derived2's version of func(): ";
        cout << i+i << '\n';
    }
};

int main()
{
    base *p;
    base ob(10);
    derived1 d_ob1(10);
    derived2 d_ob2(10);

    p = &ob;
    p->func(); // use base's func()

    p = &d_ob1;
    p->func(); // use derived1's func()

    p = &d_ob2;
    p->func(); // use derived2's func()

    return 0;
}
```

This program displays the following output:

```
Using base version of func():  10
```

```
Using derived1's version of func():  100
Using derived2's version of func():  20
```

The redefinition of a virtual function inside a derived class might, at first, seem somewhat similar to function overloading. However, the two processes are distinctly different. First, an overloaded function must differ in type and/or number of parameters, while a redefined virtual function must have precisely the same type and number of parameters and the same return type. (In fact, if you change either the number or type of parameters when redefining a virtual function, it simply becomes an overloaded function and its virtual nature is lost.) Further, virtual functions must be class members. This is not the case for overloaded functions. Also, while destructor functions can be virtual, constructors cannot. Because of the difference between overloaded functions and redefined virtual functions, the term *overriding* is used to describe virtual function redefinition.

As you can see, the example program creates three classes. The **base** class defines the virtual function **func()**. This class is then inherited by both **derived1** and **derived2**. Each of these classes overrides **func()** with its individual implementation. Inside **main()**, the base class pointer **p** is declared along with objects of type **base**, **derived1**, and **derived2**. First, **p** is assigned the address ob **ob** (an object of type **base**). When **func()** is called by using [, it is the version in **base** that is used. Next, **p** is assigned the address of **d_ob1** and **func()** is called again. Because it is the type of the object pointed to that determines which virtual function will be called, this time it is the overridden version in **derived1** that is executed. Finally, **p** is assigned the address of **d_ob2** and **func()** is called again, This time, it is the version of **func()** defined inside **derived2** that is executed.

The key points to understand from the preceding example are that the type of the object being pointed to determines which version of an overridden virtual function will be executed when accessed via a base class pointer, and that this decision is made at run time.

2. Virtual functions are hierarchical in order of inheritance. Further, when a derived class does *not* override a virtual function, the function defined within its base class is used. For example, here is a slightly different version of the preceding program:

```
// Virtual functions are hierarchical.
#include <iostream>
using namespace std;

class base
{
public:
    int i;
    base(int x) { i = x; }
    virtual void func()
    {
        cout << "Using base version of func(): ";
        cout << i << '\n';
    }
};

class derived1 : public base
{
public:
```

```
    derived1(int x) : base(x) {}
    void func()
    {
        cout << "Using derived1's version of func(): ";
        cout << i*i << '\n';
    }
};

class derived2 : public base
{
public:
    derived2(int x) : base(x) {}
    // derived2 does not override func()
};

int main()
{
    base *p;
    base ob(10);
    derived1 d_ob1(10);
    derived2 d_ob2(10);

    p = &ob;
    p->func(); // use base's func()

    p = &d_ob1;
    p->func(); // use derived1's func()

    p = &d_ob2;
    p->func(); // use base's func()

    return 0;
}
```

This program displays the following output:

```
Using base version of func():  10
Using derived1's version of func():  100
Using base version of func():  10
```

In this version, **derived2** does not override **func()**. When **p** is assigned **d_ob2** and **func()** is called, **base**'s version is used because it is next up in the class hierarchy. In general, when a derived class does not override a virtual function, the base class's version is used.

3. The next example shows how a virtual function can respond to random events that occur at run time. This program selects between **d_ob1** and **d_ob2** based upon the value returned by the standard random number generator **rand()**. Keep in mind that the version of **func()** executed is resolved at run time. (Indeed, it is impossible to resolve the calls to **func()** at compile time.)

```
/*
   This example illustrates how a virtual function
   can be used to respond to random events occurring
```

```
   at run time.
*/
#include <iostream>
#include <cstdlib>
using namespace std;

class base
{
public:
    int i;
    base(int x) { i = x; }
    virtual void func()
    {
        cout << "Using base version of func(): ";
        cout << i << '\n';
    }
};

class derived1 : public base
{
public:
    derived1(int x) : base(x) {}
    void func()
    {
        cout << "Using derived1's version of func(): ";
        cout << i*i << '\n';
    }
};

class derived2 : public base
{
public:
    derived2(int x) : base(x) {}
    void func()
    {
        cout << "Using derived2's version of func(): ";
        cout << i+i << '\n';
    }
};

int main()
{
    base *p;
    derived1 d_ob1(10);
    derived2 d_ob2(10);
    int i, j;

    for(i=0; i<10; i++)
    {
        j = rand();
        if((j%2))
```

```
            p = &d_ob1; // if odd use d_ob1
        else
            p = &d_ob2; // if even use d_ob2
        p->func(); // call appropriate function
    }

    return 0;
}
```

4. Here is a more practical example of how a virtual function can be used. This program creates a generic base class called **area** that holds two dimensions of a figure. It also declares a virtual function called **getarea()** that, when overridden by derived classes, returns the area of the type of figure defined by the derived class. In this case, the declaration of **getarea()** inside the base class determines the nature of the interface. The actual implementation is left to the classes that inherit it. In this example, the area of a triangle and a rectangle are computed.

```
// Use virtual function to define interface.
#include <iostream>
using namespace std;

class area
{
    double dim1, dim2; // dimensions of figure
public:
    void setarea(double d1, double d2)
    {
        dim1 = d1;
        dim2 = d2;
    }
    void getdim(double &d1, double &d2)
    {
        d1 = dim1;
        d2 = dim2;
    }
    virtual double getarea()
    {
        cout << "You must override this function\n";
        return 0.0;
    }
};

class rectangle : public area
{
public:
    double getarea()
    {
        double d1, d2;
        getdim(d1, d2);
        return d1 * d2;
    }
};
```

```
class triangle : public area
{
public:
    double getarea()
    {
        double d1, d2;

        getdim(d1, d2);
        return 0.5 * d1 * d2;
    }
};

int main()
{
    area *p;
    rectangle r;
    triangle t;

    r.setarea(3.3, 4.5);
    t.setarea(4.0, 5.0);

    p = &r;
    cout << "Rectangle has area: " << p->getarea() << '\n';

    p = &t;
    cout << "Triangle has area: " << p->getarea() << '\n';

    return 0;
}
```

Notice that the definition of **getarea()** inside **area** is just a placeholder and performs no real function. Because **area** is not linked to any specific type of figure, there is no meaningful definition that can be given to **getarea()** inside **area**. In fact, **getarea()** must be overridden by a derived class in order to be useful. In the next section, you will see a way to enforce this.

EXERCISES

1. Write a program that creates a base class called **num**. Have this class hold an integer value and contain a virtual function called **shownum()**. Create two derived classes called **outhex** and **outoct** that inherit **num**. Have the derived classs override **shownum()** so that it displays the value in hexadecimal and octal, respectively.

2. Write a program that creates a base class called **dist** that stores the distance between two points in a **double** variable. In **dist**, create a virtual function called **trav_time()** that outputs the time it takes to travel that distance, assuming that the distance is in miles and the speed is 60 miles per hour. In a derived class called **metric**, override **trav_time()** so that it outputs the travel time assuming that the distance is in kilometers and the speed is 100 kilometers per hour.

10.3 MORE ABOUT VIRTUAL FUNCTIONS

As Example 4 from the preceding section illustrates, sometimes when a virtual function is declared in the base class there is no meaningful operation for it to perform. This situation is common because often a base class does not define a complete class by itself. Instead, it simply supplies a core set of member functions and variables to which the derived class supplies the remainder. When there is no meaningful action for a base class virtual function to perform, the implication is that any derived class *must* override this function. To ensure that this will occur, C++ supports *pure virtual functions.*

A pure virtual function has no definition relative to the base class. Only the function's prototype is included. To make a pure virtual function, use this general form:

```
virtual type func_name(parameter_list)=0;
```

The key part of this declaration is the setting of the function equal to 0. This tells the compiler that no body exists for this function relative to the base class. When a virtual function is made pure, it forces any derived class to override it. If a derived class does not, a compile-time error results. Thus, making a virtual function pure is a way to guarantee that a derived class will provide its own redefinition.

When a class contains at least one pure virtual function, it is referred to as an *abstract class.* Since an abstract class contains at least one function for which no body exists, it is, technically, an incomplete type, and no objects of that class can be created. Thus, abstract classes exist only to be inherited. They are neither intended nor able to stand alone. It is important to understand, however, that you can still create a pointer to an abstract class, since it is through the use of base class pointers that run-time polymorphism is achieved. (It is also permissible to have a reference to an abstract class.)

When a virtual function is inherited, so is its virtual nature. This means that when a derived class inherits a virtual function from a base class and then the derived class is used as a base for yet another derived class, the virtual function can be overridden by the final derived class (as well as the first derived class). For example, if base class B contains a virtual function called **f()**, and D1 inherits B and D2 inherits D1, both D1 and D2 can override **f()** relative to their respective classes.

EXAMPLES

1. Here is an improved version of the program shown in Example 4 in the predeing section. In this version, the function **getarea()** is declared as pure in the base class **area**.

```
// Create an abstract class
#include <iostream>
using namespace std;

class area
{
    double dim1, dim2; // dimensions of figure
public:
    void setarea(double d1, double d2)
    {
        dim1 = d1;
        dim2 = d2;
    }
    void getdim(double &d1, double &d2)
    {
        d1 = dim1;
```

```
        d2 = dim2;
    }
    virtual double getarea() = 0; // pure virtual function
};

class rectangle : public area
{
public:
    double getarea()
    {
        double d1, d2;
        getdim(d1, d2);
        return d1 * d2;
    }
};

class triangle : public area
{
public:
    double getarea()
    {
        double d1, d2;

        getdim(d1, d2);
        return 0.5 * d1 * d2;
    }
};

int main()
{
    area *p;
    rectangle r;
    triangle t;

    r.setarea(3.3, 4.5);
    t.setarea(4.0, 5.0);

    p = &r;
    cout << "Rectangle has area: " << p->getarea() << '\n';

    p = &t;
    cout << "Triangle has area: " << p->getarea() << '\n';

    return 0;
}
```

Now that **getarea()** is pure, it ensures that each derived class will override it.

2. The following program illustrates how a function's virtual nature is preserved when it is inherited:

```
// Virtual functions retain their virtual nature when
   inherited,
```

```
#include <iostream>
using namespace std;

class base
{
public:
    virtual void func()
    {
        cout << "Using base version of func()\n";
    }
};

class derived1 : public base
{
public:
    void func()
    {
        cout << "Using derived1's version of func()\n";
    }
};

// derived2 inherits derived1.
class derived2 : public derived1
{
public:
    void func()
    {
        cout << "Using derived2's version of func()\n";
    }
};

int main()
{
    base *p;
    base ob;
    derived1 d_ob1;
    derived2 d_ob2;

    p = &ob;
    p->func(); // use base's func()

    p = &d_ob1;
    p->func(); // use derived1's func()

    p = &d_ob2;
    p->func(); // use derived2's func()

    return 0;
}
```

In this program, the virtual function **func()** is first inherited by **derived1**, which overrides it relative to itself. Next, **derived2** inherits **derived1**. In **derived2**, **func()** is again

overridden.

Because virtual functions are hierarchical, if **derived2** did not override **func()**, when **d_ob2** was accessed, **derived1**'s **func()** would have been used. If neither **derived1** nor **derived2** had overridden **func()**, all references to it would have been routed to the one defined in **base**.

EXERCISES

1. On your own, experiment with the two example programs. Specifically, try creating an object by using **area** from Example 1 and observe the error message. In Example 2, try removing the redefinition of **func()** within **derived2**. Confirm that, indeed, the version inside **derived1** is used.

2. Why can't an object be created by using an abstract class?

3. In Example 2, what happens if you remove only the redefinition of **func()** inside **derived1**? Does the program still compile and run? If so, why?

10.4 APPLYING POLYMORPHISM

Now that you know how to use a virtual function to achieve run-time polymorphism, it is time to consider how and why to use it. As has been stated many times in this book, polymorphism is the process by which a common interface is applied to two or more similar (but technically different) situations, thus implementing the "one interface, multiple methods" philosophy. Polymorphism is important because it can greatly simplify complex systems. A single, well-defined interface is used to access a number of different but related actions, and artificial complexity is removed. In essence, polymorphism allows the logical relationship of similar actions to become apparent; thus, the program is easier ti understand and maintain. When related actions are accessed through a common interface, you have less to remember.

There are two terms that are often linked to OOP in general and to C++ specifically. They are *early binding* and *late binding*. It is important that you know what they mean. Early binding essentially refers to those events that can be known at compile time. Specifically, it refers to those function calls that can be resolved during compilation. Early bound entities include "normal" functions, overloaded functions, and non-virtual member and friend functions. When these types of functions are compiled, all address information necessary to call them is known at compile time. The main advantage of early binding (and the reason that it is so widely used) is that it is very efficient. Calls to functions bound at compile time are the fastest types of function calls. The main disadvantage is lack of flexibility.

Late binding refers to events that must occur at run time. A late bound function call is one in which the address of the function to be called is not known until the program runs. In C++, a virtual function is a late bound object. When a virtual function is accessed via a base class pointer, the program must determine at run time what type of object is being pointed to and then select which version of the overridden function to execute. The main advantage of late binding is flexibility at run time. Your program is free to respond to random events without having to contain large amounts of "contingency code." Its primary disadvantage is that there is more overhead associated with a function call. This generally makes such calls slower than those that occur with early binding.

Because of the potential efficiency trade-offs, you must decide when it is appropriate to use early binding and when to use late binding.

EXAMPLES

1. Here is a program that illustrates "one interface, multiple methods." It defines an abstract list class for integer values. The interface to the list is defined by the pure virtual functions **store()** and **retrieve()**. To store a value, call **retrieve(**. The base class **list** does not define any default methods for these actions. Instead, each derived class defines exactly what type of list will be maintained. In the program, two types of lists are implemented: a queue and a stack. Although the two lists operate completely differently, each is accessed using the same interface. You should study this program carefully.

```
// Demonstrate virtual functions.
#include <iostream>
#include <cstdlib>
using namespace std;

class list
{
public:
    list *head; // pointer to start of list
    list *tail; // pointer to end of list
    list *next; // pointer to next item
    int num; // value to be stored

    list() { head = tail = next = NULL; }
    virtual void store(int i) = 0;
    virtual int retrieve() = 0;
};

// Create a queue-type list.
class queue : public list
{
public:
    void store(int i);
    int retrieve();
};

void queue::store(int i)
{
    list *item;

    item = new queue;
    if(!item)
    {
        cout << "Allocation error.\n";
        exit(1);
    }
    item->num = i;

    // put on end of list
    if(tail)
        tail->next = item;
    tail = item;
    item->next =  NULL;
```

```
    if(!head)
        head = tail;
}

int queue::retrieve()
{
    int i;
    list *p;

    if(!head)
    {
        cout << "List empty.\n";
        return 0;
    }

    // remove from start of list
    i = head->num;
    p = head;
    head = head->next;
    delete p;

    return i;
}

// Create a stack-type list.
class stack : public list
{
public:
    void store(int i);
    int retrieve();
};

void stack::store(int i)
{
    list *item;

    item = new stack;
    if(!item)
    {
        cout << "Allocation error.\n";
        exit(1);
    }
    item->num = i;

    // put on front of list for stack-like operation
    if(head)
        item->next = head;
    head = item;
    if(!tail)
        tail = head;
}
```

```
int stack::retrieve()
{
    int i;
    list *p;

    if(!head)
    {
        cout << "List empty.\n";
        return 0;
    }

    // remove from start of list
    i = head->num;
    p = head;
    head = head->next;
    delete p;

    return i;
}

int main()
{
    list *p;

    // demonstrate queue
    queue q_ob;
    p = &q_ob; // point to queue

    p->store(1);
    p->store(2);
    p->store(3);

    cout << "Queue: ";
    cout << p->retrieve();
    cout << p->retrieve();
    cout << p->retrieve();

    cout << '\n';

    // demonstrate stack
    stack s_ob;
    p = &s_ob; // point to stack

    p->store(1);
    p->store(2);
    p->store(3);

    cout << "Stack: ";
    cout << p->retrieve();
    cout << p->retrieve();
```

```
    cout << p->retrieve();

    cout << '\n';

    return 0;
}
```

2. The **main()** function in the list program just shown simply illustrates that the list classes do, indeed, work. However, to begin to see why run-time polymorphism is so powerful, try using this **main()** instead:

```
int main()
{
    list *p;
    queue q_ob;
    stack s_ob;
    char ch;
    int i;

    for(i=0; i<10; i++)
    {
        cout << "Stack or Queue? (S/Q): ";
        cin >> ch;
        ch = tolower(ch);
        if(ch=='q')
            p = &q_ob;
        else
            p = &s_ob;
        p->store(i);
    }

    cout << "Enter T to terminate\n";
    for(;;)
    {
        cout << "Remove from Stack or Queue? (S/Q): ";
        cin >> ch;
        ch = tolower(ch);
        if(ch=='t')
            break;
        if(ch=='q')
            p = &q_ob;
        else
            p = &s_ob;
        cout << p->retrieve() << '\n';
    }

    cout << '\n';

    return 0;
}
```

This **main()** illustrates how random events that occur at run time can be easily handled by using virtual functions and run-time polymorphism. The program executes a **for** loop

running from 0 to 9. Each iteration through the loop, you are asked to choose into which type of list-stack or the queue-you want to put a value. According to your answer, the base pointer **p** is set to point to the correct object and the current value of **i** is stored. Once the loop is finished, another loop begins that prompts you to indicate you to indicate from which list to remove a value. Once again, it is your response that determines which list is selected.

While this example is trivial, you should be able to see how run-time to polymorphism can simplify a program that must respond to random events. For instance, the Windows operating system interfaces to a program by sending it messages. As far as the program is concerned, these messages are generated at random, and you r program must respond to each one as it is received. One way to respond to these messages is through the use of virtual functions.

EXERCISES

1. Add another type of list to the program in Example 1. Have this version maintain a sorted list (in ascending order). Call this list **sorted**.
2. On you r own, think about ways in which you can apply run-time polymorphism to simplify the solutions to certain types of problems.

SKILLS CHECK

Mastery Skills Check

At this point you should be able to perform the following exercises and answer the questions.

1. What is a virtual function?
2. What types of functions cannot be made virtual?
3. How does a virtual function help achieve run-time polymorphism? Be specific.
4. What is a pure virtual function?
5. What is an abstract class? What is a polymorphic class?
6. Is the following fragment correct? If not, why not?

```
class base
{
public:
    virtual int(int a) = 0;
    // ...
};

class derived : public base
{
public:
    int f(int a, int b) { return a*b; }
    // ...
};
```

7. Is the virtual quality inherited?
8. On your own, experiment with virtual functions at this time. This is an important concept and you should master the technique.



Cumulative Skills Check

This section checks how well you have integrated material in this chapter with that from the preceding chapters.

1. Enhance the list example from Section 10.4, Example 1, so that it overloads the + and − operators. Have the + store an element and the − retrieve an element.

2. How do virtual functions differ from overloaded functions?

3. On your own, reexamine some of the function overloading examples presented earlier in this book. Determine which can be converted to virtual functions. Also, think about ways in which a virtual function can solve some of your own programming problems.